임베디드 시스템

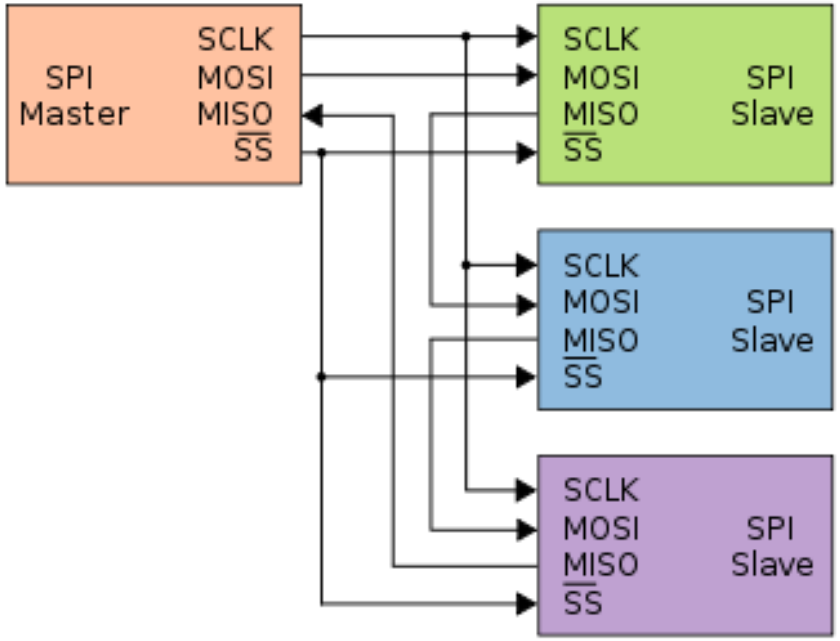
Embedded Systems

Potentiometer | CDS

SPI (1/2)

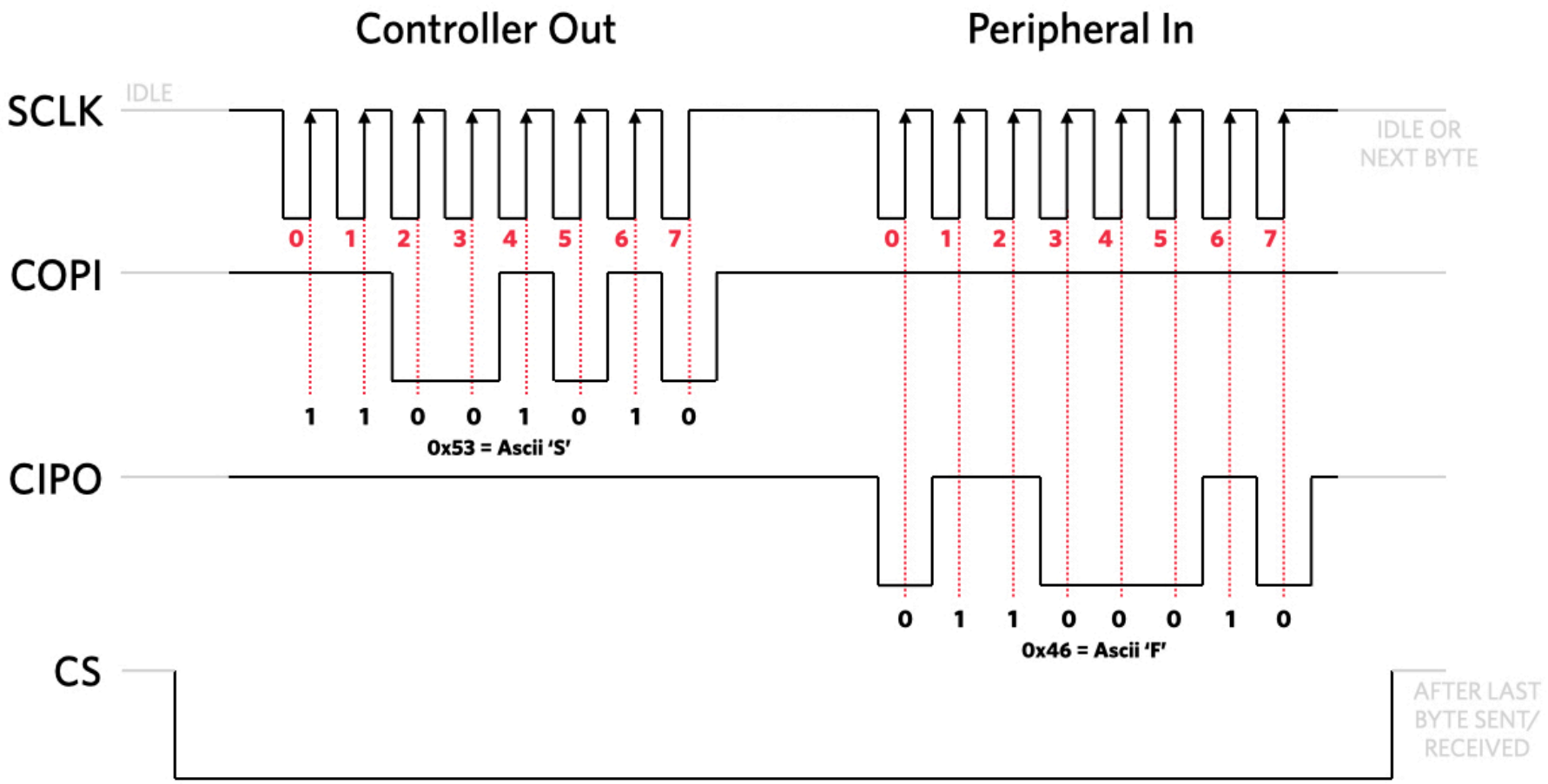
SPI(Serial Peripheral Interface)

- 시리얼 통신의 일종으로 주변 기기와의 데이터 교환을 위한 방식
- 전이중 통신이며 동기식 방식
- I2C 방식보다는 빠르나 마스터 장치는 하나만 사용할 수 있다.
- SCLK(SCL): Clock 신호를 발생시키는 핀
- MOSI(Master Output Slave Input): Master -> Slave
- MISO(Master Input Slave Output): Slave -> Master
- SS(Slave Select): Slave를 선택하기 위한 핀
    - I2C에서 사용하는 slave address를 사용하지 않음



SPI (2/2)

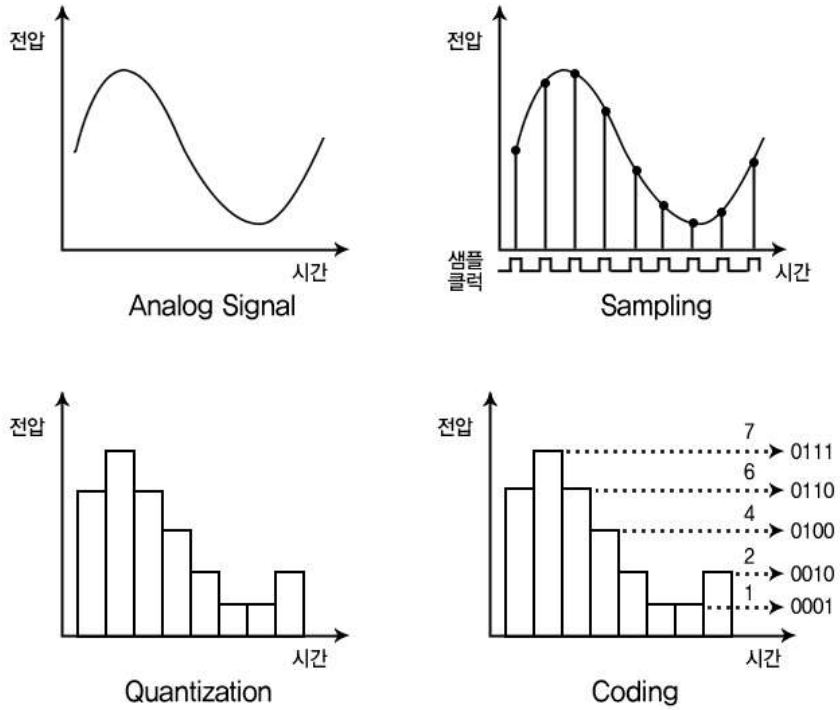
SPI 통신 원리



Potentiometer(1/4)

ADC(Analog-to-Digital Converter)

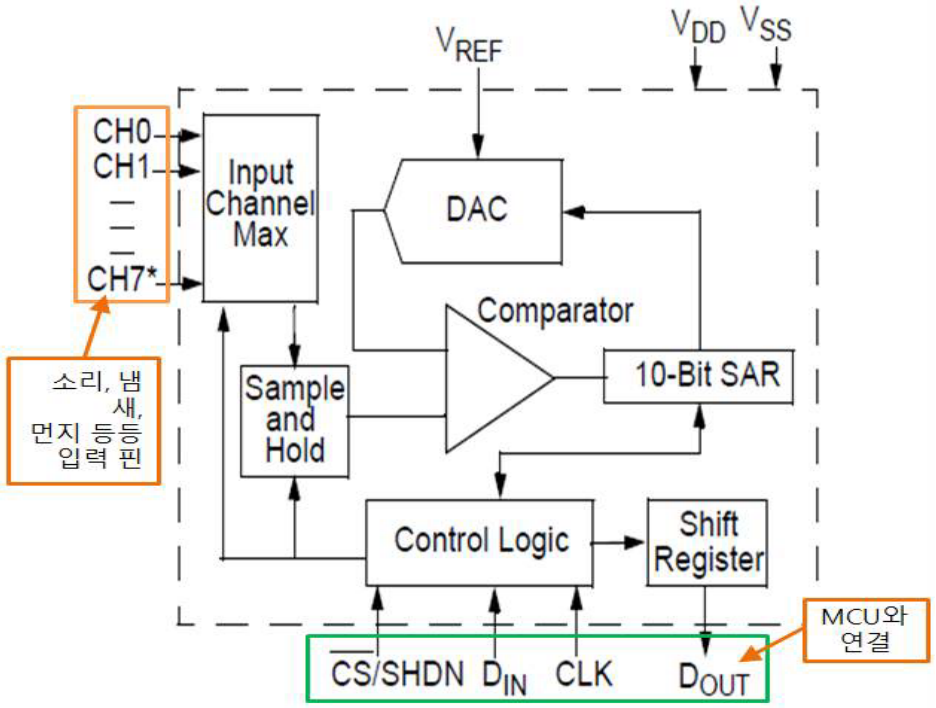
- 연속적인 값을 가지는 아날로그 신호를 디지털 신호로 변환하는 장치
- 다음과 같은 절차를 통해 변환된다.
- 표본화(Sampling)
  - 아날로그 입력 시 일정 시간 간격으로 표본을 추출한 것
  - 일정 시간 간격이 짧을수록 아날로그 입력 신호에 근접한 모양의 표본 추출 가능
  - 단위 : SPS (Sample Per Second / Sampling Rate)
- 양자화(Quantization)
  - 시간에 따른 신호의 세기를 이산화 시켜 표시하는 것
  - 이산화 값의 범위가 클수록 정밀도가 높으며 이것을 해상도(Resolution)이라 한다.
  - 8bit (0~ 255), 10bit (0 ~ 1023)
- 부호화(Coding)
  - 표본화 된 데이터를 디지털 값으로 맵핑(mapping)하여 사용하는 것

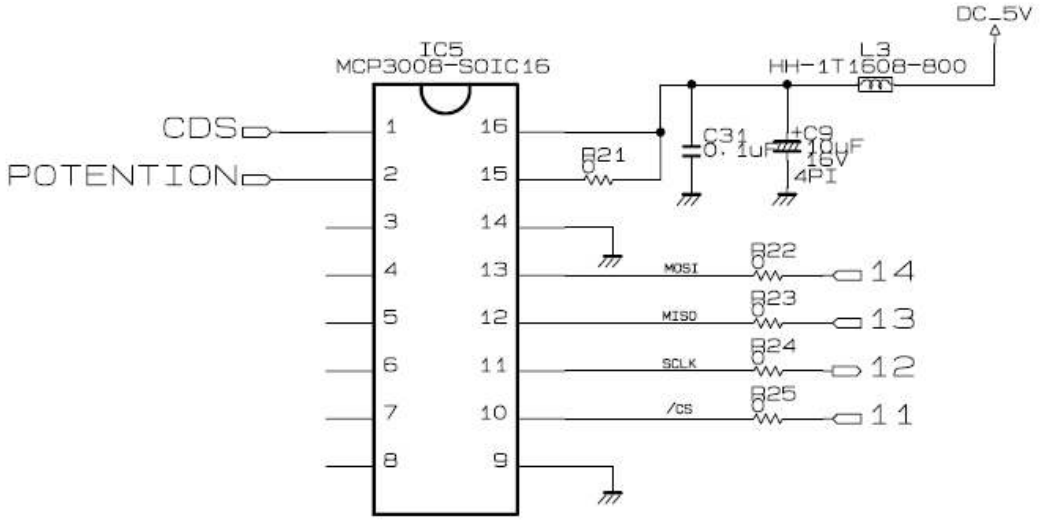


Potentiometer(2/4)

MCP3008 IC

- 아날로그 데이터를 SPI방식으로 입력 받는 집적회로

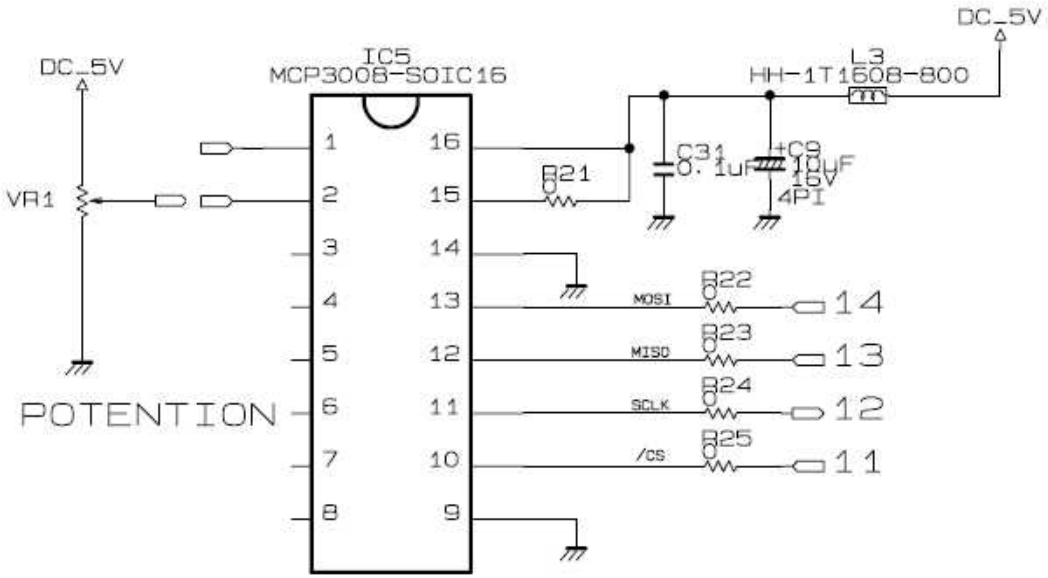




Potentiometer(3/4)

가변저항(Potentiometer)

- 전자회로의 저항 값을 정해진 범위 내에서 임의로 바꿀 수 있는 저항



		3.3v			17 || 18	0	IN	GPIO. 5	5	24
10	12	MOSI	ALT0	0	19 || 20			0v		
9	13	MISO	ALT0	0	21 || 22	0	IN	GPIO. 6	6	25
11	14	SCLK	ALT0	0	23 || 24	1	OUT	CE0	10	8
		0v			25 || 26	1	OUT	CE1	11	7
0	30	SDA.0	IN	0	27 || 28	0	IN	SCL.0	31	1
5	21	GPIO.21	OUT	0	29 || 30			0v		
6	22	GPIO.22	IN	0	31 || 32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33 || 34			0v		
19	24	GPIO.24	IN	0	35 || 36	0	OUT	GPIO.27	27	16
26	25	GPIO.25	IN	0	37 || 38	0	IN	GPIO.28	28	20
		0v			39 || 40	0	IN	GPIO.29	29	21
BCM	wPi	Name	Mode	V	Physical		V	Mode	Name	wPi	BCM

Pi 3B+

Potentiometer(4/4)

```c
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>
#include <errno.h>
#include <string.h>
#define CHAN_CONFIG_SINGLE 8 //- 싱글 채널 설정 값
#define SPI_SPEED 1000000 //- 1MHz
#define SPI_CE_CHANNEL 1 //- SPI와 연결된 CE 채널
#define SPI_ADC_POTEN_CHANNEL 1 //- Potentionmeter 연결 채널
#define CS_MCP3208 11 //- MCP3208 연결 핀
int spiSetup(void);
int readAnalogData(int adcChannel);
int main(void){
      if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }
    //- SPI통신 초기화
    spiSetup();
    //- ADC IC MCP3008 초기화
    pinMode(CS_MCP3208, OUTPUT);
    int adcValue = 0;
      while(1){
        //- 센서 값 읽기
        adcValue = readAnalogData(SPI_ADC_POTEN_CHANNEL);
        printf(" Potentionmeter Value = %d\n", adcValue);
        delay(500);
    }
    return 0;
}
```

```c
int spiSetup(void){
    if( wiringPiSPISetup (SPI_CE_CHANNEL, SPI_SPEED) == -1 ){
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s\n",
strerror (errno));
        return 1;
    }
}
//- 채널 1번에서 측정된 ADC센서 읽어오기 ----------------------------
----------------
int readAnalogData(int adcChannel){
    int adcValue = 0;
    unsigned char buffer[3] = {0};
    //- MCP3008의 8개 채널 중 하나인지 체크
    if(adcChannel<0 || adcChannel>7) return -1;
    //- SPI 통신 버퍼 설정
    buffer[0] = 0x01;
    buffer[1] = (CHAN_CONFIG_SINGLE+adcChannel) << 4;
    //- SPI 통신할 칩 활성화
    digitalWrite(CS_MCP3208, LOW);
    //- 채널 1에서 측정된 ADC 센서 읽어 오기
    wiringPiSPIDataRW(SPI_CE_CHANNEL, buffer, 3);
    //- 읽어온 값에서 10비트 데이터 추출하기
    adcValue = ( (buffer[1] & 3 ) << 8 ) + buffer[2];
    //- SPI 통신할 칩 비활성화
    digitalWrite(CS_MCP3208, HIGH);
    return adcValue;
}
```

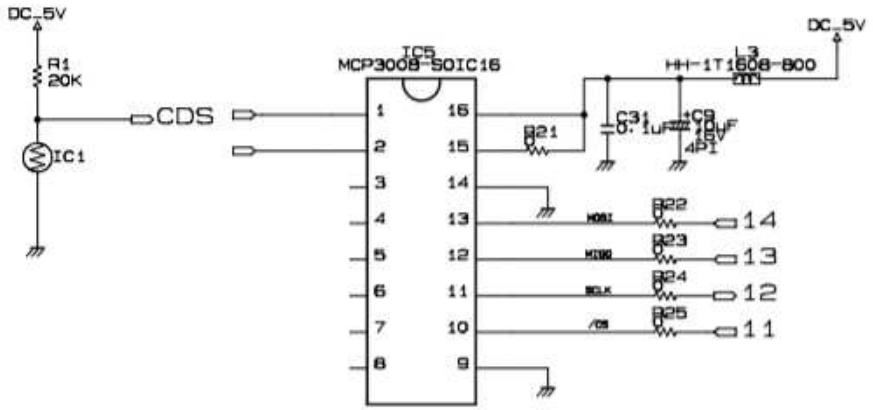
CDS (1/2)

조도(Light)센서

- 광전도효과를 이용하여 주위의 빛의 양을 측정하여 출력해주는 센서
- CDS(Cadmium sulfide): 황화 카드뮴
- 빛이 센서에 조사되면 광전도효과에 의해 내부저항이 감소한다.
- 빛의 세기와 저항 값은 반비례 관계가 성립한다.

BCM	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	BCM
		3.3v			17 || 18	0	IN	GPIO. 5	5	24
10	12	MOSI	ALT0	0	19 || 20			0v		
9	13	MISO	ALT0	0	21 || 22	0	IN	GPIO. 6	6	25
11	14	SCLK	ALT0	0	23 || 24	1	OUT	CE0	10	8
		0v			25 || 26	1	OUT	CE1	11	7
0	30	SDA.0	IN	0	27 || 28	0	IN	SCL.0	31	1
5	21	GPIO.21	OUT	0	29 || 30			0v		
6	22	GPIO.22	IN	0	31 || 32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33 || 34			0v		
19	24	GPIO.24	IN	0	35 || 36	0	OUT	GPIO.27	27	16
26	25	GPIO.25	IN	0	37 || 38	0	IN	GPIO.28	28	20
		0v			39 || 40	0	IN	GPIO.29	29	21

Pi 3B



CDS (2/2)

```c
#include <stdio.h>
#include <wiringPi.h>
#include <wiringPiSPI.h>
#include <errno.h>
#include <string.h>
#define CHAN_CONFIG_SINGLE 8 //- 싱글 채널 설정 값
#define SPI_SPEED 1000000 //- 1MHz
#define SPI_CE_CHANNEL 1 //- SPI와 연결된 CE 채널
#define SPI_ADC_LIGHT_CHANNEL 0 //- Light 연결 채널
#define CS_MCP3208 11 //- MCP3208 연결 핀
int spiSetup(void);
int readAnalogData(int adcChannel);
int main(void){
    //- wiringPi 초기화
    if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }
    //- SPI통신 초기화
    spiSetup();
    //- ADC IC MCP3008 초기화
    pinMode(CS_MCP3208, OUTPUT);
    int adcValue = 0;
    //- 센서 읽어서 출력
    while(1){
    //- 센서 값 읽기
    adcValue = readAnalogData(SPI_ADC_LIGHT_CHANNEL);
    printf(" Light Value = %dLux\n", adcValue);
    delay(500);
    }
    return 0;
}
```

```c
int spiSetup(void){
    if( wiringPiSPISetup (SPI_CE_CHANNEL, SPI_SPEED) == -1 ){
        fprintf (stderr, "wiringPiSPISetup Failed ! ERROR : %s\n",
strerror (errno));
        return 1;
    }
}
//- 채널 1번에서 측정된 ADC센서 읽어오기 ----------------------------
----------------
int readAnalogData(int adcChannel){
    int adcValue = 0;
    unsigned char buffer[3] = {0};
    //- MCP3008의 8개 채널 중 하나인지 체크
    if(adcChannel<0 || adcChannel>7) return -1;
    //- SPI 통신 버퍼 설정
    buffer[0] = 0x01;
    buffer[1] = (CHAN_CONFIG_SINGLE+adcChannel) << 4;
    //- SPI 통신할 칩 활성화
    digitalWrite(CS_MCP3208, LOW);
    //- 채널 1에서 측정된 ADC 센서 읽어 오기
    wiringPiSPIDataRW(SPI_CE_CHANNEL, buffer, 3);
    //- 읽어온 값에서 10비트 데이터 추출하기
    adcValue = ( (buffer[1] & 3 ) << 8 ) + buffer[2];
    //- SPI 통신할 칩 비활성화
    digitalWrite(CS_MCP3208, HIGH);
    return adcValue;
}
```

과제 11

가변저항과 조도센서를 이용한 금고를 만든다.

- 초기에는 아무 일도 일어나지 않는다.
- enter 키를 누르면 시스템이 시작되고, 10~1000 사이의 정수형 난수를 생성하여 키 값으로 저장한다.
- 가변 저항의 값이 화면상에 주기적으로 출력된다.
- 가변 저항의 값이 키 값의 ± 100 범위에 들어오면 빨간색 LED가 켜진다.
- 가변 저항의 값이 키 값의 ± 10 범위에 들어오면 수동부저로 상승 음계를 출력한다. 이 때 금고의 문이 열린다.
- 금고의 문이 열렸을 때, 조도가 정상이면 아무일도 일어나지 않는다.
- 조도가 낮으면(사람의 손으로 가렸을 때 도달하는 값) 카메라를 이용하여 사진을 촬영한다.
- 모든 작업이 수행되면 프로그램이 종료된다.

과제 12

가변저항을 이용하여 전자 하모니카를 만드시오. (23-2 중간고사 기출문제)

- 초기에는 어떠한 동작을 수행하지 않고, enter를 누르면 시스템이 시작된다.
- 가변 저항의 값은 수동부저의 주파수를 나타낸다. 가변 저항이 왼쪽 끝에 있을 때 주파수는 대략 200이고, 오른쪽 끝에 있을 때는 주파수가 대략 700이다.
- 키보드 Q를 누르면 해당 주파수의 음을 수동 부저를 통해 0.2초 동안 출력한다.
- 키보드 W를 누르면 해당 주파수의 음을 멈추지 않고 출력한다.
- 부저가 울릴 때는 빨간색 LED가 켜진다.
- 키보드 R을 누르면 수동 부저가 정지하고 LED가 꺼진다.

고생하셨습니다

Thank you!